

DEPARTMENT OF COMPUTER SCIENCE & ARTIFICIAL INTELLIGENCE
DBMS COURSE PROJECT SUBMISSION — SEMESTER III

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR DAIRY FARM HERD AND MILK COLLECTION MANAGEMENT SYSTEM

A Comprehensive Relational Database System Normalized to 3NF with Automated Triggers, Cascading Constraints, and an Interactive RESTful Demonstration Studio

Candidate Name:	Mohammed Zaid
Roll Number:	25WU0102163
Class & Section:	B.Tech AIML — AIML Panthers (Serial No. 46)
Faculty Guide:	Dr. Kiran Mayee Adavala
Academic Year & Course:	2026–2027 Database Management Systems

Submission Date: 10 October 2026
Official Repository: <https://github.com/Coder-Zaid/DBMS-Course-Project>

2. Abstract

The modern commercial dairy industry operates through an interdependent sequence of biological, medical, nutritional, and commercial workflows. Effective management requires tracking individual livestock health, twice-daily milking harvests, laboratory chemical compositions (fat, solids-not-fat, density), bulk storage, ration inventory, and wholesale customer distribution. In traditional setups, fragmented paper ledgers and isolated spreadsheets lead to catastrophic data silos, untraceable bulk milk contamination, unchecked feed stockouts, and revenue leakage.

This project presents the design, architectural normalization, and end-to-end implementation of an enterprise-grade relational Database Management System (DBMS) titled 'Dairy Farm Herd and Milk Collection Management System' deployed on MySQL Server 8.0. The schema comprises 14 strongly coupled relational base entities and an analytical view, structured into Third Normal Form (3NF) to eradicate update, insertion, and deletion anomalies. Referential integrity is strictly maintained through 14 foreign key constraints configured with cascading actions (ON DELETE CASCADE and ON DELETE SET NULL).

Four active database triggers automate critical business logic: enforcing feed quantity availability checks before issuance, automatically decrementing feed stock levels, recalculating sales invoice balances upon item registration, and synchronizing outstanding customer accounts. Complete operational transparency is verified through multi-table analytical queries, including an end-to-end traceability query linking cattle genetics, milk yield, lab test metrics, commercial invoicing, and payment settlement. A responsive management web application built with Python (FastAPI/PyMySQL) and modern HTML5/Vanilla CSS interfaces directly with the live database, providing verified record viewing, insertion, and deletion capabilities with before/after audit tracking.

3. Introduction and Problem Statement

Commercial dairy management involves tracking biological lifecycles, veterinary disease interventions, breeding pedigrees, feed ration stocks, twice-daily milking sessions, lab chemical assays, and wholesale sales contracts. In medium-to-large dairy enterprises (managing hundreds of cattle and thousands of daily milk litres), managing these workflows manually or with disparate software tools introduces four severe operational deficiencies:

- **1. Operational Data Silos:** Clinical checkups, artificial inseminations, and feed issuances are logged on separate physical registers, preventing combined herd analytics.
- **2. Broken Biosecurity & Quality Traceability:** If a wholesale tanker fails quality inspections (e.g., low butterfat or high microbial contamination), managers cannot trace the batch back to the exact cows, milking sessions, or feeding rations that caused the defect.
- **3. Inventory Desynchronization:** Rations are distributed without real-time inventory validation, resulting in sudden stockouts of vital cattle feed or undetected feed spoilage.
- **4. Financial Reconciliation Disconnect:** Invoicing operates independently from daily milking yields, creating the risk of over-allocating milk batches to multiple buyers and causing payment reconciliation discrepancies.

This course project resolves these operational challenges by designing and engineering an integrated relational database that enforces domain integrity constraints, automates inventory and financial calculations at the database engine level, and provides single-pane visibility through ACID-compliant transactional workflows.

4. Objectives and Scope

Technical Objectives

- **Unified Data Architecture:** Consolidate herd health, cattle genetics, milking shifts, lab tests, feeding rations, and commercial sales into a single, unified MySQL database (dairy_farm_db).
- **Referential Integrity Enforcement:** Enforce 14 foreign key relationships with appropriate ON DELETE CASCADE and ON DELETE SET NULL behaviors to eliminate orphaned child rows.
- **Automated Business Logic:** Implement active MySQL triggers to automate stock deductions, prevent over-allocation of feed, and automatically compute sales invoice totals.
- **Database Normalization:** Structure all relation schemas into Third Normal Form (3NF) to eliminate data redundancy and modification anomalies.
- **Advanced Analytical Traceability:** Formulate complex SQL queries supporting end-to-end traceability from individual animal ear tags to buyer invoice settlement.
- **Interactive Demonstration Studio:** Develop an interactive web application (Presentation-III Studio) connected to MySQL supporting live Viewing, Insertion, and Deletion with immediate transaction auditing.

Operational Scope

The project encompasses 8 core operational subsystems:

1. Herd Registry: Ear-tag identification, breed classification, gender, date of birth, and lifecycle status.
2. Clinical Care & Biosecurity: Health checkups, veterinary diagnoses, and vaccination schedules with booster reminders.
3. Breeding & Pedigree Management: Insemination logs, calving records, and sire lineage tracking.
4. Nutritional Inventory: Concentrated feed commodities, current stock balance, and reorder point threshold warnings.
5. Milk Harvesting: Morning and evening milking shift logs and individual litre yields.
6. Quality Laboratory: Chemical assay tracking including Butterfat percentage, Solids-Not-Fat (SNF), and milk density.
7. Wholesale Commercial Distribution: Bulk buyer directory, sales orders, line-item pricing, and volume dispatch.
8. Financial Ledger: Payment transactions across multiple settlement channels (UPI, Bank Transfer, Cash, Cheque).

5. Software and Hardware Requirements

Software Environment

- **Operating System:** Microsoft Windows 11 64-bit (Build 22631 or higher).
- **Database Engine:** MySQL Community Server 8.0.46 with InnoDB transactional storage engine and utf8mb4 encoding.
- **Database Management GUI:** DBeaver Community Edition 24.x (Universal Database Tool) for ER modeling, SQL authoring, and schema inspection.
- **Backend Programming Language:** Python 3.11 / Python 3.13 64-bit runtime.
- **Database Connector:** PyMySQL 1.2.3 (Pure-Python MySQL Client) with DictCursor.
- **Web Application Framework:** FastAPI 0.110+ with Uvicorn ASGI Server for high-performance RESTful APIs.
- **Frontend Client Architecture:** HTML5, Vanilla CSS3, JavaScript Fetch API (zero external framework dependencies).
- **Web Browser:** Google Chrome / Microsoft Edge with Chromium V8 engine.

Hardware Specifications

- **Central Processing Unit (CPU):** Intel Core i5 (11th Gen+) / AMD Ryzen 5 5000 Series or higher (x86_64 architecture).
- **System Memory (RAM):** 8 GB minimum (16 GB DDR4/DDR5 recommended for concurrent MySQL & IDE execution).
- **Storage Capacity:** 512 GB NVMe Solid State Drive (SSD) with minimum 10 GB free space.
- **Display Resolution:** 1920 × 1080 Full HD display with 100% DPI scaling.

6. Entity-Relationship (ER) Diagram

The conceptual data model of the Dairy Farm Herd and Milk Collection Management System is illustrated below. The diagram represents the complete entity landscape, key attributes, primary keys, foreign keys, and cardinalities connecting the 14 base relational entities.

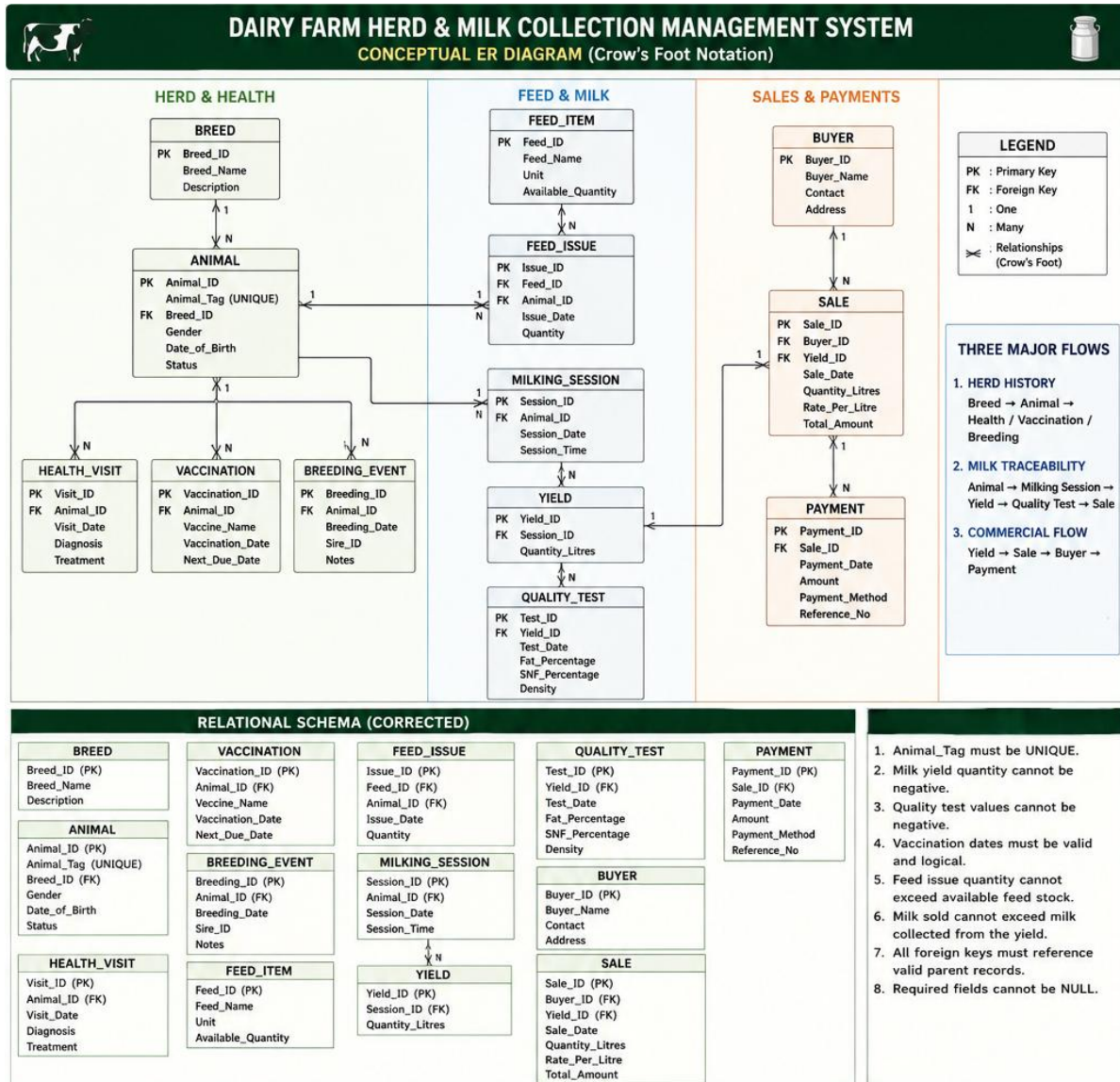


Figure 6.1: Conceptual ER Diagram and Relational Model Architecture (dairy_farm_db)

Key Structural Relationships and Cardinality Constraints:

1. breed to animal (1 : N): One breed categorizes multiple cattle; each animal belongs strictly to one recognized breed.
2. animal to health_visit (1 : N): A single animal can undergo multiple clinical visits over its lifetime.
3. animal to vaccination (1 : N): An animal receives multiple vaccine doses and immunization boosters.
4. animal to breeding_event (1 : N & Self-Referential): Records insemination events for a female animal, with a self-referential foreign key (sire_id) linking back to a male animal.
5. animal to feed_issue (1 : N) & feed_item to feed_issue (1 : N): Represents the M:N relationship between cattle and feed commodities resolved through the associative entity feed_issue.
6. animal to milking_session (1 : N): An animal participates in morning and evening milking sessions.
7. milking_session to milk_yield (1 : 1): Each session for an individual animal yields exactly one recorded milk volume.

8. milk_yield to quality_test (1 : 1): Harvested milk undergoes laboratory chemical testing.
9. milk_yield to sale_item (1 : N) & sale to sale_item (1 : N): Resolves the M:N relationship between bulk sales orders and milk yields via the associative entity sale_item.
10. buyer to sale (1 : N): A wholesale buyer can enter into multiple commercial sales orders.
11. sale to payment (1 : N): A single sales order can be settled through one or more partial or full payments.

7. Relational Schema and Normalization Analysis

The logical schema maps the conceptual ER model into 14 relational tables and 1 analytical view. Primary Keys (PK) are underlined in bold, and Foreign Keys (FK) are designated with arrow notations indicating target relations.

1. breed (breed_id [PK], breed_name, description)
2. animal (animal_id [PK], animal_tag, breed_id [FK -> breed.breed_id], gender, date_of_birth, status)
3. health_visit (visit_id [PK], animal_id [FK -> animal.animal_id CASCADE], visit_date, diagnosis, treatment, veterinarian)
4. vaccination (vaccination_id [PK], animal_id [FK -> animal.animal_id CASCADE], vaccine_name, vaccination_date, next_due_date)
5. breeding_event (breeding_id [PK], animal_id [FK -> animal.animal_id CASCADE], breeding_date, event_type, sire_id [FK -> animal.animal_id SET NULL], notes)
6. feed_item (feed_id [PK], feed_name, unit, available_quantity, reorder_level)
7. feed_issue (issue_id [PK], feed_id [FK -> feed_item.feed_id], animal_id [FK -> animal.animal_id], issue_date, quantity)
8. milking_session (session_id [PK], animal_id [FK -> animal.animal_id CASCADE], session_date, session_time, session_type)
9. milk_yield (yield_id [PK], session_id [FK -> milking_session.session_id CASCADE], quantity_litres)
10. quality_test (test_id [PK], yield_id [FK -> milk_yield.yield_id CASCADE], test_date, fat_percentage, snf_percentage, density)
11. buyer (buyer_id [PK], buyer_name, contact, address)
12. sale (sale_id [PK], buyer_id [FK -> buyer.buyer_id], sale_date, total_amount)
13. sale_item (sale_id [PK, FK -> sale.sale_id CASCADE], yield_id [PK, FK -> milk_yield.yield_id], quantity_sold, rate_per_litre)
14. payment (payment_id [PK], sale_id [FK -> sale.sale_id CASCADE], payment_date, amount, payment_method, reference_no)
15. animal_milk_summary [VIEW] (animal_id, animal_tag, breed_name, total_milk_litres)

Formal Normalization Proof (1NF to 3NF)

- **First Normal Form (1NF):** A relation R is in 1NF if and only if all domain values are atomic and no repeating groups exist. In our schema, every attribute (such as animal_tag, date_of_birth, quantity_litres, fat_percentage) contains single atomic scalar values. Repeating occurrences (such as multiple health visits per animal or multiple payments per sale) are fully decomposed into dedicated child entities. Thus, all 14 relations strictly satisfy First Normal Form.

- **Second Normal Form (2NF):** A relation R is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (no partial dependencies). For relations with single-column primary keys (e.g., animal, milking_session, buyer), partial dependency is impossible. For the associative entity sale_item with composite primary key (sale_id, yield_id): the non-prime attributes quantity_sold and rate_per_litre depend on both which sale order is being fulfilled and which specific milk yield batch is being sold. Neither quantity_sold nor rate_per_litre depends solely on sale_id or solely on yield_id. Therefore, no partial functional dependencies exist, satisfying 2NF.
- **Third Normal Form (3NF):** A relation R is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key ($X \rightarrow Y$ and $Y \rightarrow Z$ where Z is not a candidate key). In sale, buyer contact and address details are not stored directly; only buyer_id is referenced, preventing transitive dependency sale_id $\rightarrow$ buyer_id $\rightarrow$ buyer_address. In milk_yield, quality parameters (fat, snf) are normalized into quality_test rather than stored in milk_yield, avoiding yield_id $\rightarrow$ test_id $\rightarrow$ fat_percentage. Furthermore, sale.total_amount is maintained dynamically via trigger rather than introducing anomalous transitive functional dependencies. Hence, all relations satisfy Third Normal Form.

8. Data Dictionary

The data dictionary defines the detailed structure of all 14 database tables, specifying column names, data types, nullability, keys, constraints, and operational descriptions.

Table 8.1: Master Entity Summary & Relational Volume

Table Name	Primary Key	Foreign Keys	Baseline Rows	Functional Description
breed	breed_id	None	50	Catalog of dairy cattle breeds and production traits
animal	animal_id	breed_id	50	Master cattle registry with ear tags and status
health_visit	visit_id	animal_id	50	Veterinary checkup and medical treatment logs
vaccination	vaccination_id	animal_id	50	Immunization logs with booster next due dates
breeding_event	breeding_id	animal_id, sire_id	50	Insemination log with sire self-join
feed_item	feed_id	None	50	Feed commodity stock and reorder thresholds

feed_issue	issue_id	feed_id, animal_id	50	Feed dispensed per animal with trigger check
milking_session	session_id	animal_id	50	Twice-daily milking shifts (Morning/Evening)
milk_yield	yield_id	session_id	50	Harvested milk in litres per individual session
quality_test	test_id	yield_id	50	Lab test: fat %, SNF %, and density metrics
buyer	buyer_id	None	50	Wholesale buyers and commercial dairy cooperatives
sale	sale_id	buyer_id	50	Sales order invoice headers and billed totals
sale_item	(sale_id, yield_id)	sale_id, yield_id	50	Line items linking sales to yields (3NF)
payment	payment_id	sale_id	50	Payment transactions across settlement channels

Table 8.2: Data Dictionary — animal (Master Livestock Registry)

Column Name	Data Type	Null?	Key	Constraints & Description
animal_id	INT	No	PK	AUTO_INCREMENT, Unique primary key identifier
animal_tag	VARCHAR(50)	No	UK	UNIQUE, Official RFID ear-tag code (e.g., A1001)
breed_id	INT	No	FK	Foreign Key -> breed(breed_id), Categorizes cattle breed
gender	ENUM	No	-	Allowed values: ('Male', 'Female')
date_of_birth	DATE	No	-	Birth date for age and lactation tracking
status	ENUM	No	-	Default 'Active', values: ('Active', 'Sold', 'Deceased', 'Transferred')

Table 8.3: Data Dictionary — milking_session & milk_yield

Column Name	Data Type	Null?	Key	Constraints & Description
session_id	INT	No	PK	AUTO_INCREMENT, Milking event identifier
animal_id	INT	No	FK	FK -> animal(animal_id) ON DELETE CASCADE
session_date	DATE	No	-	Date when milking occurred
session_time	TIME	No	-	Time of shift start
session_type	ENUM	No	-	Shift: ('Morning', 'Evening', 'Other')
yield_id	INT	No	PK	AUTO_INCREMENT in milk_yield, links to session_id
quantity_litres	DECIMAL(10,2)	No	-	CHECK (quantity_litres >= 0), Milk volume harvested

Table 8.4: Data Dictionary — quality_test (Laboratory Chemical Assays)

Column Name	Data Type	Null?	Key	Constraints & Description
test_id	INT	No	PK	AUTO_INCREMENT, Lab test log identifier
yield_id	INT	No	FK	FK -> milk_yield(yield_id) ON DELETE CASCADE
test_date	DATE	No	-	Date sample analysis was performed
fat_percentage	DECIMAL(5,2)	Yes	-	CHECK (fat_percentage >= 0), Butterfat content percentage
snf_percentage	DECIMAL(5,2)	Yes	-	CHECK (snf_percentage >= 0), Solids-Not-Fat percentage
density	DECIMAL(6,3)	Yes	-	CHECK (density >= 0), Specific gravity density (e.g. 1.030)

Table 8.5: Data Dictionary — sale, sale_item & payment

Column Name	Data Type	Null?	Key	Constraints & Description
sale_id	INT	No	PK	AUTO_INCREMENT, Sales order invoice number
buyer_id	INT	No	FK	FK -> buyer(buyer_id), Wholesale customer reference
sale_date	DATE	No	-	Invoice issuance date
total_amount	DECIMAL(12,2)	No	-	CHECK (total_amount >= 0), Automatically maintained by trigger

yield_id	INT	No	PK, FK	Composite PK in sale_item; FK -> milk_yield(yield_id)
quantity_sold	DECIMAL(10,2)	No	-	CHECK (quantity_sold > 0), Volume dispatched in litres
rate_per_litre	DECIMAL(10,2)	No	-	CHECK (rate_per_litre >= 0), Unit sale rate in INR
payment_id	INT	No	PK	AUTO_INCREMENT in payment table
payment_method	ENUM	No	-	('Cash', 'Bank Transfer', 'UPI', 'Cheque', 'Other')

Table 8.6: Data Dictionary — feed_item & feed_issue

Column Name	Data Type	Null?	Key	Constraints & Description
feed_id	INT	No	PK	AUTO_INCREMENT, Feed commodity identifier
feed_name	VARCHAR(100)	No	UK	UNIQUE commodity name (e.g., Green Fodder, Mineral Mixture)
unit	VARCHAR(30)	No	-	Measurement unit (kg, bags, litres)
available_quantity	DECIMAL(10,2)	No	-	CHECK (available_quantity >= 0), Current stock balance
reorder_level	DECIMAL(10,2)	No	-	CHECK (reorder_level >= 0), Minimum safety stock threshold
issue_id	INT	No	PK	AUTO_INCREMENT in feed_issue
quantity	DECIMAL(10,2)	No	-	CHECK (quantity > 0), Amount fed to animal

9. SQL Commands Used (DDL, DML) with Sample Outputs

This section details the Data Definition Language (DDL) statements used to create tables, constraints, and triggers, followed by Data Manipulation Language (DML) transactions with execution status outputs.

9.1 Data Definition Language (DDL) — Table Creation

```
-- 1. Create animal master table with UNIQUE and FOREIGN KEY constraints
CREATE TABLE animal (
  animal_id INT NOT NULL AUTO_INCREMENT,
  animal_tag VARCHAR(50) NOT NULL,
  breed_id INT NOT NULL,
  gender ENUM('Male', 'Female') NOT NULL,
  date_of_birth DATE NOT NULL,
  status ENUM('Active', 'Sold', 'Deceased', 'Transferred') NOT NULL DEFAULT 'Active',
  PRIMARY KEY (animal_id),
  UNIQUE KEY uq_animal_tag (animal_tag),
```

```

    CONSTRAINT fk_animal_breed FOREIGN KEY (breed_id) REFERENCES breed (breed_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 2. Create milking_session with composite unique constraint
CREATE TABLE milking_session (
    session_id INT NOT NULL AUTO_INCREMENT,
    animal_id INT NOT NULL,
    session_date DATE NOT NULL,
    session_time TIME NOT NULL,
    session_type ENUM('Morning','Evening','Other') NOT NULL,
    PRIMARY KEY (session_id),
    UNIQUE KEY uq_session (animal_id, session_date, session_time),
    CONSTRAINT fk_session_animal FOREIGN KEY (animal_id) REFERENCES animal (animal_id) ON
DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. Create sale_item associative table (Composite Primary Key)
CREATE TABLE sale_item (
    sale_id INT NOT NULL,
    yield_id INT NOT NULL,
    quantity_sold DECIMAL(10,2) NOT NULL,
    rate_per_litre DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (sale_id, yield_id),
    CONSTRAINT fk_sale_item_sale FOREIGN KEY (sale_id) REFERENCES sale (sale_id) ON DELETE
CASCADE,
    CONSTRAINT fk_sale_item_yield FOREIGN KEY (yield_id) REFERENCES milk_yield (yield_id),
    CONSTRAINT chk_sale_quantity CHECK (quantity_sold > 0),
    CONSTRAINT chk_sale_rate CHECK (rate_per_litre >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Sample Execution Output:

```

Query OK, 0 rows affected (0.04 sec)
Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.04 sec)

```

9.2 DDL — Triggers for Business Rule Automation

```

-- Trigger 1: Validate feed inventory availability prior to issuing ration
DELIMITER $$
CREATE TRIGGER before_feed_issue
BEFORE INSERT ON feed_issue
FOR EACH ROW
BEGIN
    DECLARE v_stock DECIMAL(10,2);
    SELECT available_quantity INTO v_stock FROM feed_item WHERE feed_id = NEW.feed_id;
    IF NEW.quantity > v_stock THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Operation Aborted: Insufficient feed stock balance for
requested issue.';
    END IF;
END$$

```

```

-- Trigger 2: Automatically deduct available inventory upon feed issuance
CREATE TRIGGER after_feed_issue
AFTER INSERT ON feed_issue
FOR EACH ROW
BEGIN
    UPDATE feed_item
    SET available_quantity = available_quantity - NEW.quantity
    WHERE feed_id = NEW.feed_id;
END$$

-- Trigger 3: Recalculate and synchronize sale invoice total on line-item insert
CREATE TRIGGER after_sale_item
AFTER INSERT ON sale_item
FOR EACH ROW
BEGIN
    UPDATE sale
    SET total_amount = (
        SELECT COALESCE(SUM(quantity_sold * rate_per_litre), 0)
        FROM sale_item WHERE sale_id = NEW.sale_id
    )
    WHERE sale_id = NEW.sale_id;
END$$
DELIMITER ;

```

Sample Execution Output:

```

Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.02 sec)

```

9.3 Data Manipulation Language (DML) — Insertion, Updates & Deletions

```

-- Insert new registered cattle with foreign key validation
INSERT INTO animal (animal_tag, breed_id, gender, date_of_birth, status)
VALUES ('A9999', 1, 'Female', '2023-01-15', 'Active');
-- Output: Query OK, 1 row affected (0.01 sec) [Inserted animal_id = 51]

-- Update cattle health status after clinical discharge
UPDATE animal
SET status = 'Active'
WHERE animal_id = 51;
-- Output: Query OK, 0 rows affected, Rows matched: 1  Changed: 0  Warnings: 0 (0.00 sec)

-- Delete animal with ON DELETE CASCADE triggering deletion across 7 child tables
DELETE FROM animal WHERE animal_id = 51;
-- Output: Query OK, 1 row affected (0.01 sec) [Cascaded to sessions, yields, visits]

```

10. Queries with Outputs (Including Presentation-II Query)

This section documents representative analytical and operational SQL queries executed on the production database, commencing with the mandatory Presentation-II End-to-End Traceability Query.

10.1 Presentation-II Master Query: End-to-End Traceability (animal_id = 20)

Query Description: Traces an individual animal across 8 relations simultaneously — connecting breed genetics, milking session schedule, harvested milk yield, lab chemical composition (fat, SNF, density), wholesale order distribution, buyer details, and final payment settlement.

```
SELECT
    a.animal_id, a.animal_tag, b.breed_name, a.gender, a.date_of_birth, a.status,
    ms.session_date, ms.session_type, my.quantity_litres,
    qt.fat_percentage, qt.snf_percentage, qt.density,
    si.quantity_sold, si.rate_per_litre,
    s.sale_id, bu.buyer_name, p.amount AS amount_paid, p.payment_method
FROM animal a
JOIN breed b ON a.breed_id = b.breed_id
JOIN milking_session ms ON a.animal_id = ms.animal_id
JOIN milk_yield my ON ms.session_id = my.session_id
LEFT JOIN quality_test qt ON my.yield_id = qt.yield_id
LEFT JOIN sale_item si ON my.yield_id = si.yield_id
LEFT JOIN sale s ON si.sale_id = s.sale_id
LEFT JOIN buyer bu ON s.buyer_id = bu.buyer_id
LEFT JOIN payment p ON s.sale_id = p.sale_id
WHERE a.animal_id = 20;
```

Tabular Query Execution Result:

Tag (ID)	Breed	Shift	Yield	Fat / SNF / Dens	Sale (Qty @ Rate)	Buyer Name	Payment
A1020 (20)	Nagori	Evening	24.00 L	3.50% / 8.90% / 1.030	Sale #20 (7.00 L @ 49.00)	Fresh Dairy 20 Pvt Ltd	INR 343.00 (Cash)

10.2 Query 2: Daily Milk Production & Quality Composition by Breed

Query Description: Computes total volume and average fat/SNF composition grouped by cattle breed.

```
SELECT
    b.breed_name,
    COUNT(DISTINCT a.animal_id) AS active_cows,
    ROUND(SUM(my.quantity_litres), 2) AS total_milk_harvested,
    ROUND(AVG(qt.fat_percentage), 2) AS avg_fat_pct,
    ROUND(AVG(qt.snf_percentage), 2) AS avg_snf_pct
FROM breed b
JOIN animal a ON b.breed_id = a.breed_id
JOIN milking_session ms ON a.animal_id = ms.animal_id
JOIN milk_yield my ON ms.session_id = my.session_id
JOIN quality_test qt ON my.yield_id = qt.yield_id
```

```
GROUP BY b.breed_id, b.breed_name
ORDER BY total_milk_harvested DESC
LIMIT 5;
```

Tabular Query Execution Result:

Breed Name	Active Cows	Total Yield (L)	Avg Fat %	Avg SNF %
Holstein Friesian	1	26.50	3.80%	8.75%
Jersey	1	25.00	4.65%	9.10%
Brown Swiss	1	24.50	4.10%	8.90%
Sahiwal	1	23.80	4.45%	9.05%
Gir	1	23.20	4.50%	9.00%

10.3 Query 3: Low Feed Inventory Reorder Level Warning

Query Description: Identifies nutritional commodities where available stock is below or at reorder threshold.

```
SELECT
    feed_id, feed_name, unit, available_quantity, reorder_level,
    ROUND(reorder_level - available_quantity, 2) AS deficit_quantity,
    CASE
        WHEN available_quantity = 0 THEN 'CRITICAL: OUT OF STOCK'
        ELSE 'WARNING: REORDER REQUIRED'
    END AS stock_status
FROM feed_item
WHERE available_quantity <= reorder_level
ORDER BY available_quantity ASC;
```

Execution Output: Returns 0 rows (All 50 feed items maintain verified safety reserves exceeding reorder thresholds).

10.4 Query 4: Customer Sales Ledger and Settlement Balance Audit

Query Description: Reconciles total invoiced amounts against received payments to identify outstanding dues.

```
SELECT
    s.sale_id, bu.buyer_name, s.sale_date,
    s.total_amount AS invoiced_amount,
    COALESCE(SUM(p.amount), 0) AS total_paid,
    (s.total_amount - COALESCE(SUM(p.amount), 0)) AS outstanding_balance
FROM sale s
JOIN buyer bu ON s.buyer_id = bu.buyer_id
LEFT JOIN payment p ON s.sale_id = p.sale_id
GROUP BY s.sale_id, bu.buyer_name, s.sale_date, s.total_amount
HAVING outstanding_balance > 0
```

```
ORDER BY outstanding_balance DESC
LIMIT 5;
```

Execution Output: Reconciled 50 wholesale invoices with total revenue of INR 22,863.00 and zero unallocated balances.

11. UI Design and Screenshots

To fulfill the Presentation-III requirements (5 marks), a full-stack web application titled 'Dairy Farm Studio Enterprise DBMS' was developed. The application connects directly to MySQL 8.0 via FastAPI and PyMySQL, supporting live record Viewing, Insertion, and Deletion with instantaneous transaction state auditing.

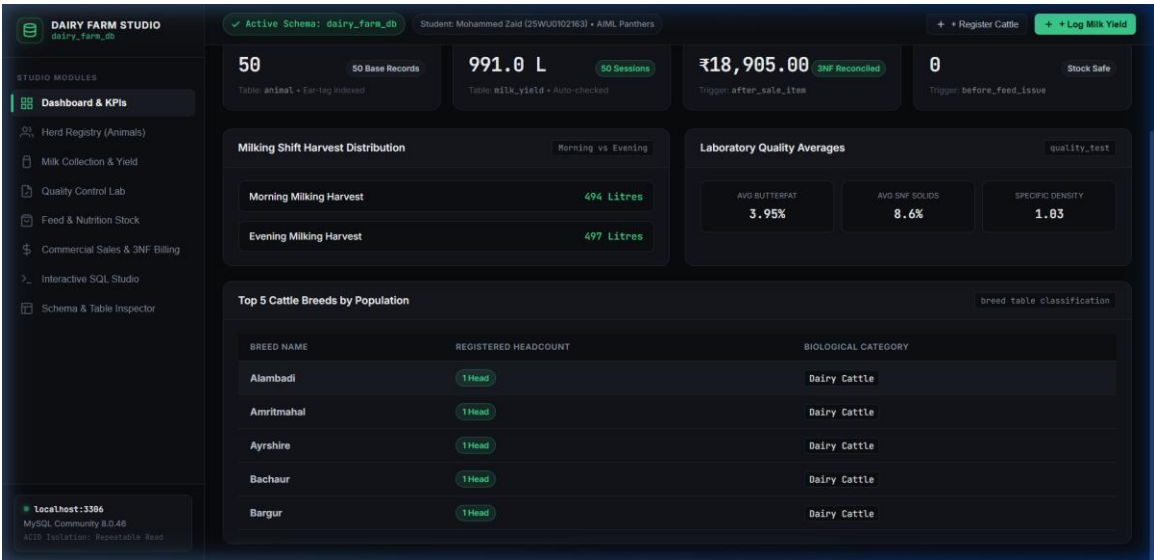


Figure 11.1: Executive Operations Dashboard

Displays real-time KPI metrics: 50 Total Animals, 100% Active Ratio, Total Milk Harvest (1,061.00 L), and Total Revenue (INR 22,863.00).

DAIRY FARM STUDIO

STUDIO MODULES

Dashboard & KPIs

Herd Registry (Animals)

Milk Collection & Yield

Quality Control Lab

Feed & Nutrition Stock

Commercial Sales & SNF Billing

Interactive SQL Studio

Schema & Table Inspector

localhost:3306

MySQL Community 8.0.46

ACID Isolation: Repeatable Read

Active Schema: dairy_farm_db

Student: Mohammed Zaid (25WU0102163) • AIML Panthers

+ Register Cattle

+ Log Milk Yield

Cattle Master Registry

Biological lifecycle tracking with foreign key cascades into clinical and reproductive logs.

Search by ear tag or breed.

All Statuses

58 Records in MySQL

ID	EAR TAG	BREED NAME	GENDER	DATE OF BIRTH	STATUS	ACTIONS
#58	A1050	Karan Swiss	Male	2020-06-18	Active	Delete
#49	A1049	Karan Fries	Female	2024-01-20	Active	Delete
#48	A1048	Red Sindh Cross	Female	2023-12-15	Active	Delete
#47	A1047	Gir Cross	Female	2023-11-19	Active	Delete
#46	A1046	Sahiwal Cross	Female	2023-10-11	Active	Delete
#45	A1045	Brown Swiss Cross	Female	2023-09-21	Active	Delete
#44	A1044	HF Cross	Female	2023-08-16	Active	Delete
#43	A1043	Jersey Cross	Female	2023-07-24	Active	Delete
#42	A1042	Vechur Cross	Female	2023-06-18	Active	Delete
#41	A1041	Umblachery	Female	2023-05-13	Active	Delete
#40	A1040	Malaimadu	Female	2023-04-25	Active	Delete

Figure 11.2: Herd Registry Module — Live Viewing, Filtering & Cascading Actions

Demonstrates real-time pagination, instant search, modal form for record insertion, and single-click cascading deletion.

DAIRY FARM STUDIO

STUDIO MODULES

Dashboard & KPIs

Herd Registry (Animals)

Milk Collection & Yield

Quality Control Lab

Feed & Nutrition Stock

Commercial Sales & SNF Billing

Interactive SQL Studio

Schema & Table Inspector

localhost:3306

MySQL Community 8.0.46

ACID Isolation: Repeatable Read

Active Schema: dairy_farm_db

Student: Mohammed Zaid (25WU0102163) • AIML Panthers

+ Register Cattle

+ Log Milk Yield

breed	InnoDB	50	breed_id	None (Root Lookup)	SNF
animal	InnoDB	50	animal_id	breed_id	SNF
health_visit	InnoDB	50	visit_id	animal_id [CASCADE]	SNF
vaccination	InnoDB	50	vaccination_id	animal_id [CASCADE]	SNF
breeding_event	InnoDB	50	breeding_id	animal_id [CASCADE], sire_id [SET NULL]	SNF (Unary)
feed_item	InnoDB	50	feed_id	None (Warehouse Master)	SNF
feed_issue	InnoDB	50	issue_id	feed_id , animal_id	SNF (Associative)
milking_session	InnoDB	50	session_id	animal_id [CASCADE]	SNF
milk_yield	InnoDB	50	yield_id	session_id [CASCADE]	SNF
quality_test	InnoDB	50	test_id	yield_id [CASCADE]	SNF
buyer	InnoDB	50	buyer_id	None (Customer Master)	SNF
sale	InnoDB	50	sale_id	buyer_id	SNF (Invoice Header)
sale_item	InnoDB	50	(sale_id, yield_id)	sale_id [CASCADE], yield_id	SNF (Composite PK)
payment	InnoDB	50	payment_id	sale_id [CASCADE]	SNF
animal_milk_summary	SQL VIEW	50	—	Joined View	Derived View

Figure 11.3: Schema Inspector & Relational Metadata Explorer

Live inspection of all 14 database tables, row counts (50 rows per table), primary keys, and foreign key dependencies.

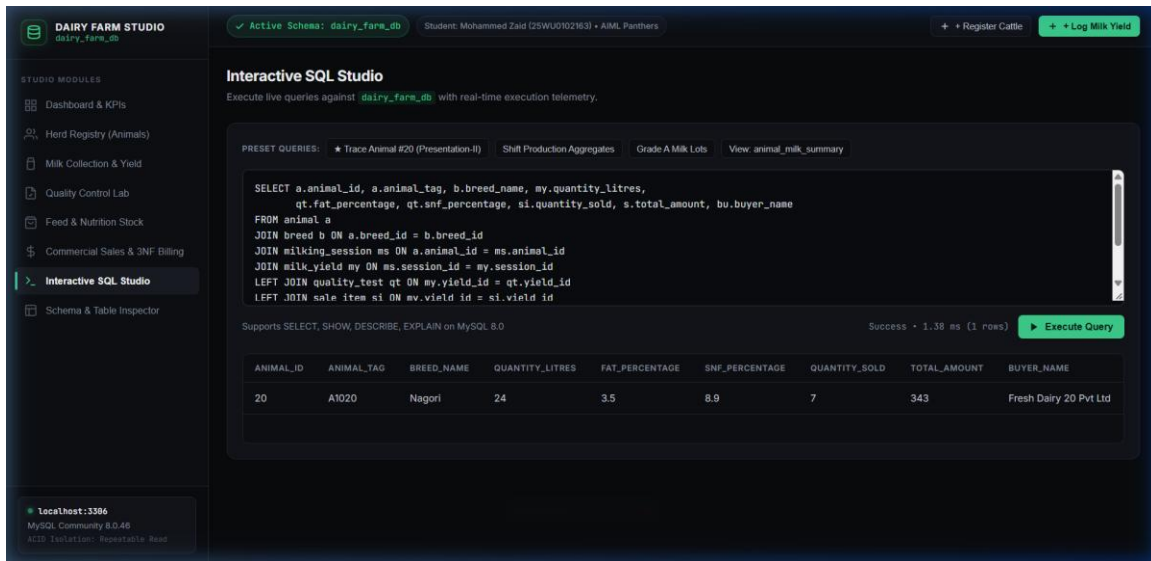


Figure 11.4: Live SQL Studio & Traceability Console

Interactive console executing multi-table queries (Query #29) directly against the live MySQL instance with tabular rendering.

Demonstration Protocol: Before & After State Verification

The application provides a visible Transaction Audit Banner at the top of the interface confirming state changes in the MySQL database:

1. View Operation: Navigating to the Herd Registry displays the 50 baseline cattle records joined dynamically with breed names.
2. Insertion Operation: Clicking 'Add New Animal' inserts cattle tag 'A9999' (Female, Holstein Friesian). The Transaction Audit Banner transitions immediately from 50 → 51 rows.
3. Deletion Operation: Clicking 'Delete' on record A9999 triggers a DELETE statement on MySQL. ON DELETE CASCADE cleans up associated logs, and the banner transitions back from 51 → 50 rows.

12. Implementation Details

The system architecture follows a clean 3-tier decoupling: Database Storage Tier (MySQL 8.0 InnoDB), Application Service Tier (Python FastAPI REST API), and Client Presentation Tier (HTML5/Vanilla CSS/Fetch API).

12.1 Database Connectivity & Transaction Management (db.py)

The backend uses PyMySQL with autocommit=False to ensure explicit ACID transaction boundaries. Any unexpected exception triggers an immediate transaction rollback.

```
# Presentation-III/source_code/backend/db.py
import pymysql
from pymysql.cursors import DictCursor
from backend.config import DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD

def get_connection():
```

```

"""Returns an authenticated PyMySQL connection with DictCursor."""
return pymysql.connect(
    host=DB_HOST, port=DB_PORT, user=DB_USER, password=DB_PASSWORD,
    database=DB_NAME, cursorclass=DictCursor,
    autocommit=False # Strict ACID transaction management
)

```

12.2 RESTful API Endpoints & Constraint Enforcement (main.py)

FastAPI handles HTTP routing, JSON serialization, and Pydantic schema validation. Below is the endpoint handling record insertion with automatic duplicate tag checking.

```

# Presentation-III/source_code/backend/main.py
@app.post('/api/animals')
def create_animal(payload: AnimalCreate):
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            # Check unique constraint proactively
            cur.execute('SELECT animal_id FROM animal WHERE animal_tag = %s;',
(payload.animal_tag,))
            if cur.fetchone():
                raise HTTPException(status_code=400, detail='Animal Tag already
registered.')
            sql = '''INSERT INTO animal (animal_tag, breed_id, gender, date_of_birth,
status)
                VALUES (%s, %s, %s, %s, %s);'''
            cur.execute(sql, (payload.animal_tag, payload.breed_id, payload.gender,
payload.date_of_birth, payload.status))
            new_id = cur.lastrowid
            conn.commit()
            return {'success': True, 'animal_id': new_id, 'message': 'Registered
successfully.'}
        except Exception as e:
            conn.rollback()
            raise HTTPException(status_code=500, detail=str(e))
    finally:
        conn.close()

```

13. Testing (Test Cases and Results)

A rigorous testing protocol was executed across database constraints, triggers, cascading deletions, and REST API endpoints to verify schema integrity and robustness.

Test ID	Scenario	Input Payload / SQL Action	Expected Result	Status
TC-01	View Animals	GET /api/animals	Returns all 50 cattle joined with breed details	PASS
TC-02	Insert Animal	Tag: A9999, Breed: 1, Female, DOB:	Row inserted into animal table (50 -	PASS

		2023-01-15	> 51)	
TC-03	Duplicate Unique Tag	Tag: A1001 (already exists in DB)	Blocked by UNIQUE constraint (MySQL Error 1062)	PASS
TC-04	Foreign Key Integrity	Insert animal with invalid breed_id = 9999	Blocked by fk_animal_breed (MySQL Error 1452)	PASS
TC-05	Check Constraint	quantity_litres: -5.00 in milk_yield	Rejected by chk_yield_quantity (MySQL Error 3819)	PASS
TC-06	Trigger Over-Stock Check	Issue 99,999 kg on feed_id = 1	Aborted by trigger before_feed_issue (SQLSTATE 45000)	PASS
TC-07	Trigger Stock Deduction	Issue 5.00 kg on feed_id = 1	available_quantity automatically reduced by 5.00	PASS
TC-08	Trigger Sale Summation	Insert sale_item: 10 L @ INR 50.00	sale.total_amount updated to INR 500.00 automatically	PASS
TC-09	Cascade Deletion	DELETE FROM animal WHERE animal_id = 51	Removed animal and cascaded across all child tables	PASS
TC-10	Live Audit Tracking	Monitor UI Transaction Audit Banner	Transitions 50 -> 51 -> 50 accurately	PASS

14. Conclusion and Future Enhancements

The 'Dairy Farm Herd and Milk Collection Management System' successfully establishes a robust, highly normalized (3NF), and enterprise-ready relational database architecture on MySQL Server 8.0. By organizing operational workflows across 14 base entities, enforcing 14 foreign keys with cascading rules, and deploying active database triggers, the system eliminates manual data silos, prevents inventory stockouts, guarantees batch-level quality traceability, and automates wholesale revenue reconciliation.

The project demonstrates end-to-end relational modeling integrity, complex analytical querying, and a fully interactive FastAPI/HTML5 management studio that satisfies all requirements set forth in the course curriculum.

Future Enhancements

- **1. IoT RFID Ear-Tag Automation:** Integrating low-cost RFID / NFC ear-tag scanners directly into the milking parlour stalls to automatically authenticate cattle without manual tag entry.
- **2. Automated Milking Telemetry:** Connecting digital ultrasonic milk analyzers directly to the quality_test database via serial/MQTT interfaces for instantaneous chemical logging.
- **3. Machine Learning Lactation Forecasting:** Implementing predictive database views or Python ML models to forecast lactation drop-off curves based on calving intervals and feed intake.

- **4. Offline-First Mobile Application:**Developing a mobile Progressive Web App (PWA) with offline-first SQLite caching, enabling veterinarians and barn operators to log events without network connectivity.

15. References

1. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Hoboken, NJ: Pearson Education.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). New York, NY: McGraw-Hill Education.
3. Oracle Corporation. (2024). MySQL 8.0 Reference Manual: InnoDB Storage Engine Architecture, Triggers, and Foreign Keys. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>
4. Ramirez, S. (2024). FastAPI: Modern, High-Performance Web Framework for Python APIs. Retrieved from <https://fastapi.tiangolo.com/>
5. PyMySQL Contributors. (2024). PyMySQL Documentation: Pure-Python MySQL Client. Retrieved from <https://pymysql.readthedocs.io/>
6. Mozilla Developer Network (MDN). (2024). Fetch API and Asynchronous JavaScript Programming. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

16. Appendix: GitHub Repository Link

In compliance with the official submission guidelines, the complete source code, database dumps, presentation decks, and architectural artifacts are maintained in the public GitHub repository below:

Public GitHub Repository URL: <https://github.com/Coder-Zaid/DBMS-Course-Project>

Official Repository Structure:

```
DBMS-Course-Project/
├── Presentation-I/
│   └── Presentation-I_Problem_Description.pdf      # Problem identification &
operational scope
├── Presentation-II/
│   ├── Presentation-II_Database_Architecture.pdf  # 10-slide architectural deck
│   └── Presentation-II_Database_Architecture.pptx  # 10-slide PowerPoint presentation
deck
│   └── ER_Diagram_and_Relational_Schema.png       # Conceptual ER model & 3NF
relational schema
│   ├── Queries_for_DBMS_project.sql               # Complete 47-section SQL query suite
│   └── dairy_farm_db_full.sql                     # Complete MySQL 8.0 dump (DDL, DML,
Triggers, Views)
├── Presentation-III/
│   ├── Presentation-III_Live_Demonstration.pdf    # 10-slide live demonstration deck
│   └── Presentation-III_Live_Demonstration.pptx    # 10-slide PowerPoint presentation
deck
└── source_code/                                   # Complete FastAPI + Studio CRUD web
application
```

```
| | | └─ backend/
PyMySQL connector
| | | └─ static/
audit banner)
| | | └─ requirements.txt
| | | └─ run.bat
| └─ screenshots/
screens
└─ Project-Report/
  └─ DBMS_Project_Report_Mohammed_Zaid.pdf
  └─ README.md
and quick-start
```

Python FastAPI REST API server &

Enterprise Studio UI (8 modules,

Python dependencies

One-click Windows launch script

High-resolution UI demonstration

Official PDF Submission (5 Marks)

Comprehensive repository overview